```
function reportData = singleImageInspectionFunction(img, useClassifier)

% Single Image Algorithm code to pre/post process TUBE images

%==========================================================================
% AI CLASSIFIER INITIALIZATION
%==========================================================================
% initialize the model with the "persistent" datatype
% which keeps the model loaded in memory between function calls
persistent aiModel;
persistent classNames;

if isempty(aiModel) && useClassifier
    % Load the AI model and class names if not already loaded
    loadedData = load("tubeClassifierNet.mat");
    aiModel = loadedData.trainedNet;

    %define the classes/classifications here
    classNames = ["colorMismatch", "defectiveLength", "good", "malformedMetal"];
end



% ========================================================================
% COLOR ANOMALY DETECTION (The Blue Tubes)
% ========================================================================
% Convert to HSV (Hue, Saturation, Value) to safely isolate the blue paint. The H and S channels are
% sufficient for this step. (V channel controls brightness)
[numColorAnomalies, colorStats] = colorAnomalyDetection(img);

% ========================================================================
% SHAPE ANOMALY DETECTION (The Silver Tubes)
% ========================================================================
[numNormalTubes, numShapeAnomalies, totalTubesInImage, shapeMask, annotatedImage] = shapeAnomalyDetection(img, colorStats, numColorAnomalies);

% ========================================================================
% AI PREDICTION & FINAL REPORT
% ========================================================================

if useClassifier
    % Preprocesses the image for resnet18 requirements (224 x 224 pixels)
    imgResized = imresize(img, [224 224]);

    % Make a prediction using the AI model with predict()
    % predict() will evaluate the image and output probability scores
    aiScores = predict(aiModel, single(imgResized));

    % Cast the scores into double variables for calculation purposes
    aiScores = double(aiScores);

    % Find the highest confidence/probability score within the array AND
    % obtain its corresponding category index
    [maxConfidence, classIdx] = max(aiScores);
    predictedCategory = classNames(classIdx);

    % Assign aiDecision outside if-else branch to avoid scope issues
    aiDecision = "";

    % Associate the given category index with the appropriate PASS/FAIL
```

```
    % rating
    if predictedCategory == "good"
        aiDecision = "PASS";
    else
        aiDecision = "FAIL";
    end

    % Convert confidence score to an equivalent percentage for the user
    confidencePercentage = maxConfidence * 100;

end

% Create a reportData structure to hold the final outputs from the AI
% and traditional image classification models
if useClassifier
    reportData.Decision = aiDecision;
    reportData.Confidence = confidencePercentage;
    reportData.AIClass = categorical(predictedCategory);
end

reportData.metrics.TotalTubes = totalTubesInImage;
reportData.metrics.NormalTubes = numNormalTubes;
reportData.metrics.ShapeAnomalies = numShapeAnomalies;
reportData.metrics.ColorAnomalies = numColorAnomalies;

reportData.images.ImageMask = shapeMask;
reportData.images.AnnotatedImage = annotatedImage;

end
```